

Kalvin Patarawong
Date: 11/01/2024
C950 Task 1

A. Identify a named self-adjusting algorithm (e.g., nearest neighbor algorithm, greedy algorithm) that could be used to create your program to deliver the packages.

The Greedy algorithm is a self-adjusting algorithm that I can utilize when I create my program to deliver the packages.

B. Identify a self-adjusting data structure, such as a hash table, that could be used with the algorithm identified in part A to store the package data.

A hash table is a self-adjusting data structure I can use to store package data alongside the algorithm.

B1. Explain how your data structure accounts for the relationship between the data components you are storing.

In the hash table, each package is stored as a key-value pair, with the package ID as the key and all relevant details (such as address, delivery deadline, and special instructions) stored as the value. This structure enables fast access to any package's information, allowing for efficient lookups, updates, and deletions based on its unique ID. By grouping each package's data in a single entry, the hash table inherently maintains the relationships between delivery requirements, making it easy to retrieve complete package information in one step. This design supports the Greedy algorithm's ability to prioritize deliveries by specific needs, which is crucial for ensuring that packages are delivered on time while optimizing the delivery route.

C1. Explain the algorithm's logic using pseudocode.

Initialize Trucks
Initialize Packages
Initialize Drivers

Assign Priority Packages

For each Truck in Trucks:

 While Truck is not full and there are priority Packages available: Determine the nearest priority
 Package to the Truck's current location Load the nearest priority Package onto the Truck
 Remove the loaded Package from the list of available Packages

Deliver Packages

For each Truck in Trucks:

 While there are Packages on the Truck:

 Deliver the next Package

 Update the Truck's current location to the Package's delivery address

While Packages are still available for delivery:

For each Truck in Trucks:

Determine the nearest Package to the Truck's current location

Load the nearest Package onto the Truck

Deliver Packages

C2. Describe the programming environment you will use to create the Python application, including the software and hardware you will use.

I will use PyCharm 2024.2.3 (Professional Edition) build #PY-242.23339.19, running on runtime version 21.0.4+13-b509.17, with Python version 3.13. My setup includes a 2021 MacBook Pro M1 Max, operating on MacOS Sequoia 15.1 and equipped with 32 GB of memory.

C3. Evaluate the space-time complexity of *each* major segment of the program and the entire program using big-O notation.

Evaluating the space and time complexity for each significant program segment is crucial for understanding its efficiency. Hash table operations, which include inserting, deleting, and retrieving packages using their unique package IDs, have a time complexity of $O(1)$ and a space complexity of $O(n)$, where n is the number of packages. Loading packages into trucks has an $O(n)$ time complexity as each truck iterates through the package list, requiring $O(1)$ space since it doesn't create additional data structures. Delivering packages also has a time complexity of $O(n)$ while maintaining a space complexity of $O(1)$. The program's time complexity is $O(n)$ due to loading and delivering packages. Its space complexity is $O(n)$ because of hash table storage, ensuring efficient handling of an increasing number of packages.

C4. Explain the capability of your solution to scale and adapt to a growing number of packages.

My programming solution is designed to be highly scalable and customizable, making it well-suited for handling a growing number of packages. The package class ensures that each package contains all the necessary information for processing, automatically rejecting any that lack the required details. Additionally, the truck class allows for the easy addition of more trucks, which increases delivery capacity. Notably, the functions and algorithms maintain consistent performance regardless of how many packages or trucks are involved, ensuring that scaling up is seamless and efficient without compromising speed or functionality.

C5. Discuss why the software design would be efficient and easy to maintain.

The software design would be efficient and easy to maintain for several reasons. First, its streamlined architecture reduces complexity, making navigation straightforward and quick modifications. The code is well-organized, with clear classes and definitions that allow for fast identification and resolution of issues. The program achieves optimal performance using a hash table with a time complexity of $O(1)$. Combining the hash table with the nearest neighbor algorithm yields faster results than other combinations. Overall, this thoughtful design

maximizes performance and makes it easier to adapt to increasing package volumes, ensuring the system remains robust and sustainable.

C6. Describe *both* the strengths and weaknesses of the self-adjusting data structure (e.g., the hash table).

The hash table has several strengths that make it a popular choice for data storage. It's swift, allowing quick data access and retrieval, and it's simple and easy to implement, mainly when working with arrays. One notable feature is its collision resolution technique called chaining, where each bucket holds a list of items to manage cases where multiple entries hash to the same index. However, there are some weaknesses to consider. For one, hash tables require extra storage space because they need to maintain a few empty slots to operate efficiently, making them less space-efficient overall.

C7. Justify the choice of a key for efficient delivery management from the following components:

The package ID is the most efficient choice for a key in delivery management. This is mainly because many packages can have the same delivery address, city, zip code, weight, status, or deadline, making those data points not unique enough to function as practical keys. Relying on non-unique identifiers could create problems in accurately getting packages to the right place on time. In contrast, the uniqueness of the package ID ensures that the correct package information is linked to its corresponding key, allowing for the development of an efficient algorithm and data structure.

D. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

Lysecky, R., & Vahid, F. (2018, June). *C950: Data Structures and Algorithms II*. zyBooks. Retrieved March 22, 2021, from <https://learn.zybooks.com/zybook/WGUC950AY20182019/>